EXPENSE SPLITTER: A SPLITWISE-STYLE GROUP EXPENSE SETTLEMENT SYSTEM USING SPRING BOOT AND DESIGN PATTERNS

A PROJECT REPORT

Submitted by

PARIKSHIT (24BET10298)
DHRUV GOUNIYAL (24BET10295)
SAGAR MAKKAR (24BET10296)
MANAN (24BET10199)

in partial fulfillment for the award of the degree of

BACHELOR OF ENGINEERING

IN

INFORMATION TECHNOLOGY



Chandigarh University

OCTOBER 2026

BONAFIDE CERTIFICATE

Certified that this project report "**EXPENSE SPLITTER: A SPLITWISE-STYLE GROUP EXPENSE SETTLEMENT SYSTEM USING SPRING BOOT AND DESIGN PATTERNS**" is the bonafide work of "**PARIKSHIT (24BET10298), DHRUV GOUNIYAL (24BET10295), SAGAR MAKKAR (24BET10296) and MANAN (24BET10199)**" who carried out the project work under my/our supervision.

SIGNATURE	**SIGNATURE**
Surinder Kaur	Harsh Gupta
HEAD OF THE DEPARTMENT	**SUPERVISOR**
Information Technology	Assistant Professor
	Information Technology

Submitted for the project viva-voce examination held on _____

INTERNAL EXAMINER **EXTERNAL EXAMINER**

TABLE OF CONTENTS

List of Figures 4
List of Tables 4
CHAPTER 1. INTRODUCTION 7
1.1. Identification of Client / Need / Relevant Contemporary Issue 7
1.2. Identification of Problem 7
1.3. Identification of Tasks 8
1.4. Timeline 9
1.5. Organization of the Report 9
CHAPTER 2. LITERATURE REVIEW/BACKGROUND STUDY 10
2.1. Timeline of the Reported Problem 10
2.2. Existing Solutions 10
2.3. Bibliometric Analysis 11
2.4. Review Summary 11
2.5. Problem Definition 12
2.6. Goals/Objectives 12
CHAPTER 3. DESIGN FLOW/PROCESS 13
3.1. Evaluation & Selection of Specifications/Features 13
3.2. Design Constraints 13
3.3. Analysis of Features and Finalization Subject to Constraints 14
3.4. Design Flow 15
3.5. Design Selection 15
3.6. Implementation Plan/Methodology 16
CHAPTER 4. RESULTS ANALYSIS AND VALIDATION 20
4.1. Implementation of Solution 20
CHAPTER 5. CONCLUSION AND FUTURE WORK 31
5.1. Conclusion 31
5.2. Future Work 32
REFERENCES 33
APPENDIX 34
A. Design Checklist 34
B. Project at a Glance 34
C. Key Source Code Excerpts 35
D. Sample API Requests and Responses 36

List of Figures

Figure 1.1 Gantt chart of the project schedule 9
Figure 3.1 Block diagram of the layered architecture 16
Figure 3.2 Entity-relationship diagram of the database 17
Figure 3.3 Class diagram of the Strategy, Observer and Singleton patterns 17
Figure 3.4 Flowchart of the transactional add-expense operation 18
Figure 4.1 Dashboard after loading the demonstration data (light theme) 23
Figure 4.2 PERCENTAGE split with the live split meter showing 10% unallocated 23
Figure 4.3 The server rejects the invalid split with 400 Bad Request 24
Figure 4.4 Expense list with per-member shares and the AuditLogger activity feed 24
Figure 4.5 Settle-up slip after a partial payment, with Settle buttons and payment history 25
Figure 4.6 Deleting a member who still has expenses returns 409 Conflict 25
Figure 4.7 Inline confirmation before deleting a group 26
Figure 4.8 Dashboard in the dark theme 26
Figure 4.9 H2 console showing the expense shares stored by Hibernate 27
Figure 4.10 Newman run of the Postman collection (last two folders and summary) 29

List of Tables

Table 2.1 Timeline of key findings related to the problem 10
Table 2.2 Analysis of reviewed literature and solutions 11
Table 3.1 Evaluation of candidate features 13
Table 3.2 Comparison of alternative designs 15
Table 4.1 Tools used in the project 20
Table 4.2 Split strategies 21
Table 4.3 Balances for the demonstration scenario 21
Table 4.4 REST endpoints 22
Table 4.5 Functional test cases and results 27
Table 4.6 Greedy settlement compared with the exact minimum 30
Table A.1 Design checklist 34
Table A.2 Technical summary of Expense Splitter 34

ABSTRACT

Shared expenses are part of everyday student life: hostel rooms, flats, trips and group outings. UPI has made paying one another instant, but it does not answer the question that causes most disputes: who owes whom, and how much. Tracking this in chat messages or spreadsheets is error-prone, uneven splits are tedious to compute, and settling up pair by pair needs far more payments than necessary. This project presents **Expense Splitter**, a Splitwise-style REST backend with a web interface that records group expenses, computes each member's net balance, proposes a short list of payments that settles the group and records those payments as they are made.

The system is built with Java 17, Spring Boot 3.3, Spring Web, Spring Data JPA and Hibernate on an H2 in-memory database, with a ready MySQL configuration. It follows a layered controller–service–repository architecture and applies three design patterns: **Strategy** for the three split methods (EQUAL, PERCENTAGE and EXACT), implemented as Spring beans injected into a Map<String, SplitStrategy> and chosen at run time from the request; **Observer** for the side effects of adding an expense (an AuditLogger that persists an activity log and a NotificationService); and **Singleton** for a ConfigService holding the currency symbol and rounding rules. Adding an expense is one @Transactional unit of work, so an invalid split rolls back the expense, its shares and its log together.

Net balances are computed with Java Streams (groupingBy and summingDouble) and settlements with a greedy algorithm over two PriorityQueue max-heaps of creditors and debtors, which needs at most n − 1 payments for n members. Money is rounded to two decimals with BigDecimal, and any paisa left over by rounding is assigned to one participant so that shares always add up exactly. A @ControllerAdvice handler turns custom exceptions into clean JSON errors with the correct HTTP status codes.

The application was validated with a JUnit integration test and a Postman collection of 41 requests and 59 assertions executed with Newman, all of which passed. Testing also showed that the greedy settlement, although never worse than n − 1 payments, is not always the true minimum (it matched the optimum in 100% of random four-member groups but 75.7% of eight-member groups), a known property of the hard debt-simplification problem that is reported openly and addressed in future work.

Keywords: *Spring Boot, REST API, Spring Data JPA, design patterns, Strategy, Observer, Singleton, transaction management, greedy algorithm, expense sharing.*

CHAPTER 1.
INTRODUCTION

1.1. Identification of Client / Need / Relevant Contemporary Issue

Shared spending is routine for students and young professionals. Hostel and PG roommates split rent, electricity and groceries; friends split trips, cab rides and restaurant bills; project teams split printing and travel costs. Each of these creates a running record of small debts between several people.

Digital payments have made the act of paying trivial. India's Unified Payments Interface (UPI), launched by NPCI in 2016, lets anyone transfer money instantly from a phone [1]. But payment apps answer "how do I pay?", not "how much do I owe, and to whom?". The bookkeeping that comes before the payment is still done in chat messages, notes or spreadsheets, which leads to forgotten expenses, arithmetic mistakes and awkward disputes between friends.

Dedicated apps such as Splitwise address this need [2], and their popularity shows that the need is real. They are, however, closed, account-based services that keep users' financial history on remote servers, and they do not let a student see how balances and settlements are actually computed.

There is also an academic need. The course *Project Based Learning in Java* (Experiment 3.3) asks students to build a complete backend that visibly applies Spring Boot, JPA/Hibernate, REST design, exception handling, transactions, Java Streams and object-oriented design patterns. An expense splitter fits well: its domain is familiar, yet it needs non-trivial relationships, validation, careful money arithmetic and a real algorithm.

The clients for this project are therefore **groups of students and young professionals** who share costs and want a transparent, private ledger, and **course instructors and students** who need a realistic, well-structured reference application.

1.2. Identification of Problem

Groups that share expenses face four connected problems:

- **Bookkeeping errors.** Records kept in chats or spreadsheets are easily lost, duplicated or mis-added, and nobody has a single trusted view of the group's balances.
- **Uneven splits.** Real expenses are not always equal: one person may have a larger room, skip a dish or pay a fixed amount. Computing percentage and exact splits by hand, and keeping the shares consistent with the total, is tedious.
- **Too many transactions.** If every debt is settled pair by pair, a group of n people can need up to $n(n-1)/2$ payments, many of which cancel each other out.
- **Money arithmetic.** Binary floating-point cannot represent most decimal amounts exactly [3], so naive code produces totals such as 999.99 for a Rs 1000 bill split three ways.

The broad problem is that there is no simple, transparent and correct way for a group to record shared expenses with different split rules, see who owes whom, and settle up with as few payments as possible.

1.3. Identification of Tasks

The work was divided into the following tasks, which also define the structure of this report:

1. **Problem identification and literature review** — study existing expense-sharing tools, the debt-settlement problem, money arithmetic, REST and enterprise design patterns (Chapters 1–2).
2. **Requirement and feature selection** — derive features and evaluate them against cost, privacy, ethical and technical constraints (Sections 3.1–3.3).
3. **Design** — compare alternative designs, select one, and model the architecture, the database and the expense-processing flow (Sections 3.4–3.6).
4. **Implementation** — build the entities and repositories, the REST API, the split strategies, the transactional expense flow, the balance and settlement engine, settle-up payments, observers, the configuration singleton, exception handling and a web interface (Chapter 4).
5. **Testing and validation** — write a JUnit integration test and a Postman collection with automated assertions, run it with Newman, and verify rollback, error codes and settlement results (Chapter 4).

6. **Documentation** — prepare this report, the README, a Postman test guide and a viva cheat-sheet.

1.4. Timeline

The project was planned over eight weeks. Figure 1.1 shows the schedule as a Gantt chart; the split-strategy and transaction work and the settlement algorithm are highlighted because they carried the most technical risk.

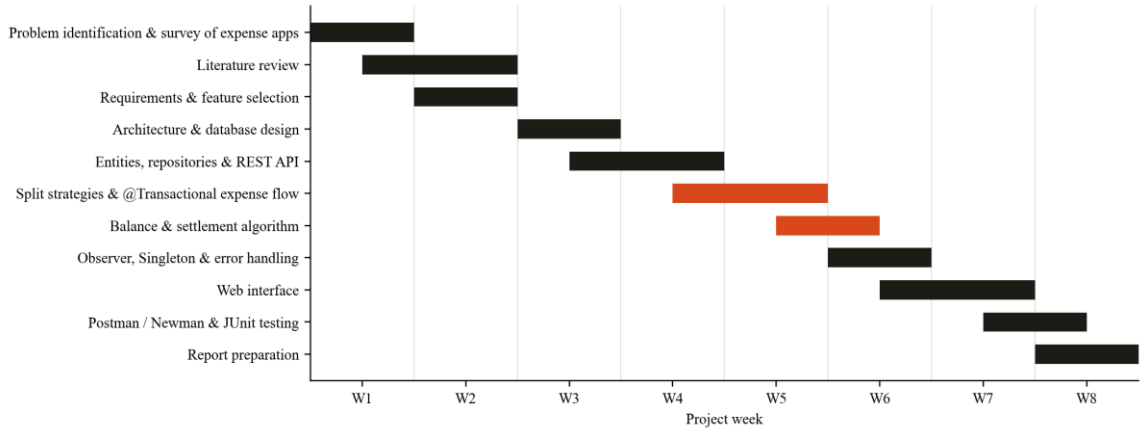


Figure 1.1 Gantt chart of the project schedule

1.5. Organization of the Report

- **Chapter 1 – Introduction** establishes the need, defines the problem, lists the tasks and presents the timeline.
- **Chapter 2 – Literature Review** traces how the problem has been studied, reviews existing solutions, analyses the literature and ends with a precise problem definition and measurable objectives.
- **Chapter 3 – Design Flow/Process** evaluates candidate features, lists design constraints, compares two alternative designs, selects one and presents the architecture, database design and algorithms.
- **Chapter 4 – Results Analysis and Validation** describes the implementation module by module, shows the working application and reports the outcome of the tests.
- **Chapter 5 – Conclusion and Future Work** summarises the outcomes, deviations from the plan and directions for extension.

CHAPTER 2.
LITERATURE REVIEW/BACKGROUND STUDY

2.1. Timeline of the Reported Problem

Table 2.1 Timeline of key findings related to the problem

Year	Milestone
1970	Codd proposes the relational model of data, the basis of SQL databases such as H2 and MySQL [5].
1985	IEEE 754 standardises binary floating-point arithmetic [4].
1991	Goldberg explains why binary floating-point cannot represent most decimal fractions exactly [3].
1994	Gamma et al. catalogue 23 object-oriented design patterns, including Strategy, Observer and Singleton [6].
2000	Fielding defines the REST architectural style for web APIs [7].
2002	Fowler describes Service Layer, Repository and Unit of Work patterns [8]; Johnson's lightweight J2EE design leads to the Spring Framework [9].
2004	Spring Framework 1.0 is released and Fowler names the Dependency Injection pattern [10]; Verhoeff analyses how to settle multiple debts efficiently and shows that finding the fewest payments is a hard problem [11].
2006	The Java Persistence API (JPA) 1.0 standardises object-relational mapping in Java [12].
2011	Splitwise launches, popularising app-based group expense sharing [2].
2014	Spring Boot 1.0 introduces auto-configuration and embedded servers [13].
2016	NPCI launches UPI, making instant person-to-person payments widespread in India [1].
2022	Spring Boot 3.0 moves to a Java 17 baseline and the Jakarta EE namespace [13].

2.2. Existing Solutions

Existing approaches to managing shared expenses fall into five groups:

- **Dedicated expense-sharing apps (Splitwise, Tricount, Settle Up).** They track group expenses with equal and unequal splits and suggest simplified repayments. They are polished, but need an account and an internet connection, store financial history on the provider's servers and do not expose how results are calculated.
- **UPI payment apps with split features.** Some payment apps let a group split a single bill and send payment requests. This works for one bill, but is tied to one payment ecosystem and does not keep a multi-expense ledger with running balances.
- **Spreadsheets.** Flexible and free, but every group must build its own sheet, formulas break easily, nothing checks that shares add up, and settlement must be worked out by hand.

- **Chat messages and notes.** The most common approach in practice: zero setup, but no structure, no totals, no history and frequent disputes.
- **Accounting software.** Accurate double-entry bookkeeping, but designed for businesses and far too heavy for a group of friends.

2.3. Bibliometric Analysis

Table 2.2 Analysis of reviewed literature and solutions

Source / solution	Key features	Effectiveness	Drawbacks
Gamma et al. [6]	Catalogue of reusable patterns incl. Strategy, Observer, Singleton	Shared vocabulary; makes variation points explicit	Overuse adds indirection; Singleton can hide global state
Fowler [8]; Johnson [9]	Service layer, Repository, Unit of Work; lightweight IoC containers	Separates business rules from persistence; testable	Needs discipline; frameworks add a learning curve
Fielding [7]	Resource URLs, uniform interface, stateless requests, status codes	Simple, language-neutral APIs	No transaction semantics across requests
Verhoeff [11]	Formal analysis of settling debts within a group	n − 1 payments always suffice	True minimum is hard to find; heuristics needed
Goldberg [3]; IEEE 754 [4]	Analysis of binary floating-point error	Explains rounding errors in money calculations	double needs explicit rounding for money
Splitwise and similar apps [2]	Group ledger, uneven splits, debt simplification	Widely used and convenient	Accounts, remote data, closed algorithms

2.4. Review Summary

The literature establishes four points that shape this project. First, a layered design with a service layer and repositories, managed by an IoC container, keeps business rules testable and separate from persistence [8], [9], [10]. Second, Strategy and Observer are the standard ways to make “many ways to split” and “many reactions to an event” extensible without long if-else chains [6]. Third, money must be handled with explicit rounding because binary floating-point is inexact [3]. Fourth, settling a group’s debts never needs more than n − 1 payments, but guaranteeing the absolute minimum is computationally hard [11], so a fast greedy heuristic is the practical choice for an interactive system.

Expense Splitter responds directly: it exposes a REST API [7] built on Spring Boot and JPA, uses Strategy, Observer and Singleton where they fit naturally, rounds money with BigDecimal, and settles debts with a two-heap greedy algorithm whose behaviour is measured and documented in Chapter 4.

2.5. Problem Definition

What is to be done: build a REST backend, with a simple web interface, through which a group can register members, record expenses paid by one member and split among several using EQUAL, PERCENTAGE or EXACT rules, view each member's net balance, obtain a short list of payments that settles the group, record those payments as they happen, and delete expenses, payments, users and groups safely.

How it is to be done: in Java 17 with Spring Boot 3, Spring Web and Spring Data JPA/Hibernate on H2 (with MySQL as an option); a layered controller–service–repository architecture; the Strategy, Observer and Singleton patterns; @Transactional expense creation; Java Streams for balances; a PriorityQueue-based greedy settlement; @ControllerAdvice error handling; and testing with JUnit and Postman/Newman.

What is not to be done: no real money movement or payment-gateway integration; no user authentication or multi-tenant security (a single trusted group in a lab setting); no currency conversion; no mobile app; and no claim that the settlement is always the mathematical minimum.

2.6. Goals/Objectives

1. Provide REST endpoints to create users and groups, add members, add, list, filter and delete expenses, view balances and settlements, record and undo payments, and delete users and groups, each returning the correct HTTP status code.
2. Implement EQUAL, PERCENTAGE and EXACT splitting as interchangeable Spring beans selected at run time, with validation that percentages total 100 and exact amounts total the expense.
3. Guarantee atomicity: an expense, its shares and its activity log are saved together or not at all.
4. Compute net balances that always sum to zero, correct to two decimal places.
5. Produce a settlement that needs at most n − 1 payments for n members with non-zero balances.
6. Notify independent listeners (audit log, notification) whenever an expense is added or deleted, without coupling them to the expense service.
7. Validate the system with automated tests: a JUnit integration test and a Postman collection with at least 50 assertions covering success and error paths.

CHAPTER 3.
DESIGN FLOW/PROCESS

3.1. Evaluation & Selection of Specifications/Features

Table 3.1 lists the features suggested by the literature and existing solutions, with a judgement of whether each is required in Expense Splitter.

Table 3.1 Evaluation of candidate features

Feature	Justification	Decision
Users, groups and membership	Core domain; many-to-many relationship	Required
Expenses with payer and participants	Core domain	Required
EQUAL / PERCENTAGE / EXACT splits	Real expenses are often uneven	Required (Strategy pattern)
Net balance per member	One trusted view of who owes whom	Required (Streams)
Simplified settlement	Avoid $n(n-1)/2$ pairwise payments [11]	Required (greedy, $\leq n-1$)
Record settle-up payments	Balances must reflect money already paid back	Required (partial payments, undo)
Activity log and notifications	Transparency and audit trail	Required (Observer)
Atomic expense creation	No half-saved expenses	Required (@Transactional)
Clean error responses	REST conventions [7]	Required (@ControllerAdvice)
Delete expenses, users, groups	Correct mistakes and clean up	Required (with integrity rules)
Web interface	Demonstration without tools	Added
User accounts / login	Needed for public deployment	Deferred (lab scope)
Real UPI payments	Convenience	Rejected (regulation, security)
Multi-currency, receipts, recurring bills	Convenience	Deferred to future work

3.2. Design Constraints

3.2.1. Standards and Constraints Considered

- **Economic / cost:** only free, open-source software (OpenJDK 17, Spring Boot, Hibernate, H2, Maven, Postman free tier); runs on a student laptop with no paid hosting. A Maven Wrapper means Maven itself need not be installed.

- **Regulations / privacy:** the system only records who paid what; it never moves money and stores no bank, card or UPI details. Data stays in a local database, and nothing beyond a name and an e-mail address is collected.
- **Ethical:** money calculations must be transparent and fair. Every share is stored and visible, rounding remainders follow a documented rule, payments cannot exceed what is owed, and a user who still appears in expenses cannot be deleted silently (409 Conflict), because that would change other members' balances.
- **Safety / reliability:** adding an expense is a single database transaction; input is validated before it is accepted; wrong IDs, amounts and split types produce clear messages instead of server errors.
- **Professional standards:** REST conventions for URLs, methods and status codes [7]; layered architecture [8]; Jakarta Persistence annotations; constructor injection as recommended by Spring [14]; plain getters and setters (no Lombok) so that every line is visible to beginners.
- **Manufacturability / portability:** pure Java 17 + Maven; the same code runs on Windows, macOS and Linux; switching from H2 to MySQL requires only configuration changes.
- **Accessibility:** the web interface supports keyboard use with visible focus, offers light and dark themes, and never relies on colour alone (balances are signed numbers with labels).

3.3. Analysis of Features and Finalization Subject to Constraints

- **Removed:** real UPI payment integration and user login (conflict with lab scope, regulation and security).
- **Modified:** balances were first planned as a stored column updated on every expense; they are instead computed on demand from expenses, shares and payments with Streams, so they can never drift out of sync after deletes.
- **Modified:** money stays in double columns for simplicity, but every computed amount passes through ConfigService.round() (BigDecimal, HALF_UP), and split remainders are assigned to one participant so shares always sum exactly.
- **Modified:** the reserved SQL words *user* and *group* could not be table names, so the tables are named users and expense_groups.

- **Modified:** deletion rules — deleting a group cascades to its expenses, shares, payments and log inside one transaction; deleting a user who still appears in expenses or payments is refused with 409 Conflict.
- **Added:** settle-up payments with partial amounts and undo, a single-page web interface with a live API console and a split meter, and a Postman collection that cleans up after itself.

3.4. Design Flow

Two complete alternative designs were considered.

Design A — Servlet and JDBC with conditional logic. A few servlets handle all requests, SQL is written by hand through JDBC, and the split method is chosen with an if-else or switch on the split type. Balances are stored in a column and updated manually on every change, and transactions are committed manually with connection.commit().

Design B — Layered Spring Boot application with JPA and design patterns (selected). Controllers only translate HTTP into method calls; services hold the business rules; Spring Data repositories provide persistence; split methods are Strategy beans in a Map; side effects are Observer listeners; transactions are declared with @Transactional; and errors are mapped centrally by @ControllerAdvice.

3.5. Design Selection

Table 3.2 Comparison of alternative designs

Criterion	Design A: Servlet + JDBC	Design B: Spring Boot + JPA
Boilerplate	High: manual SQL, connections, JSON parsing	Low: JpaRepository, Jackson, auto-configuration
Adding a new split type	Edit the if-else chain (breaks Open/Closed)	Add one @Component class; nothing else changes
Transactions	Manual commit/rollback; easy to forget	Declarative @Transactional; automatic rollback
Error handling	try-catch in every handler	One @ControllerAdvice for the whole API
Consistency of balances	Stored column can drift after deletes	Computed from source data every time
Testability	Logic mixed with HTTP and SQL	Services tested with @SpringBootTest
Fit with course syllabus	Partial	Complete — Spring Boot, JPA, IoC/DI, patterns, Streams
Learning curve	Lower at first	Higher, but mirrors industry practice

Design B was selected. It satisfies every required feature in Table 3.1, keeps business rules isolated and testable, and demonstrates every concept the experiment asks for.

3.6. Implementation Plan/Methodology

Figure 3.1 shows the layered architecture. Clients (the web interface and Postman) call controllers; controllers delegate to services; services use the Strategy, Observer and Singleton components and the repositories; Hibernate maps entities to tables in H2 or MySQL.

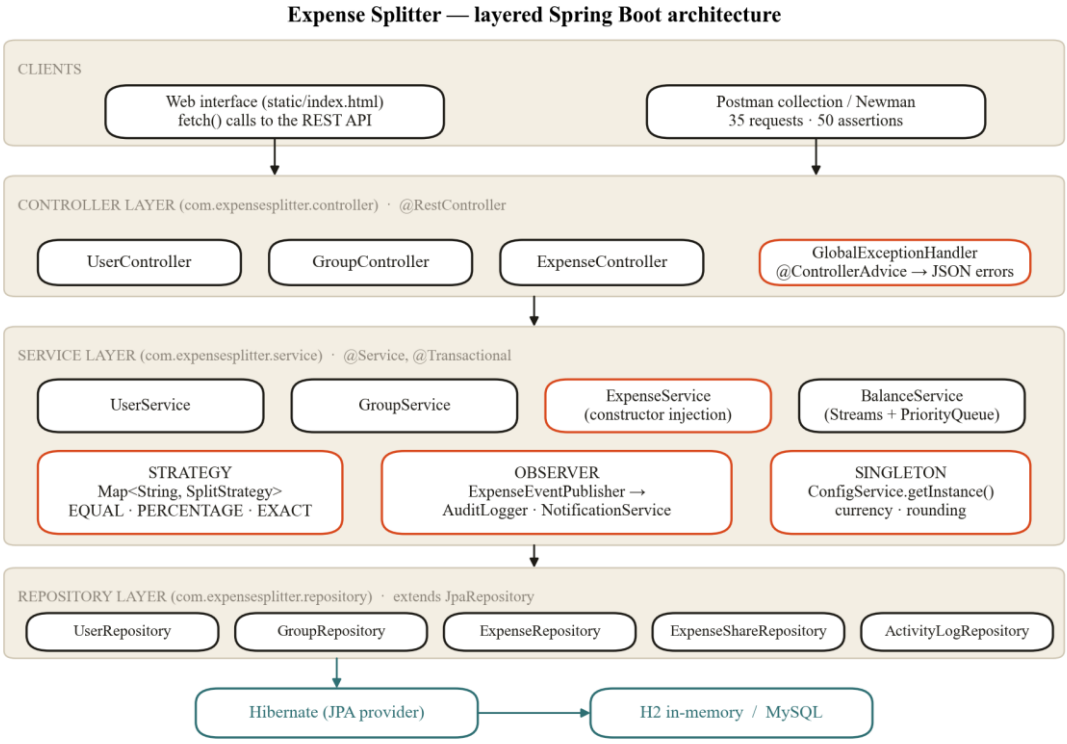


Figure 3.1 Block diagram of the layered architecture

Figure 3.2 shows the database design generated by Hibernate from the entity classes. The join table group_members implements the many-to-many link between users and groups; expense_shares stores how much each person owes for each expense; payments stores settle-up payments between members.

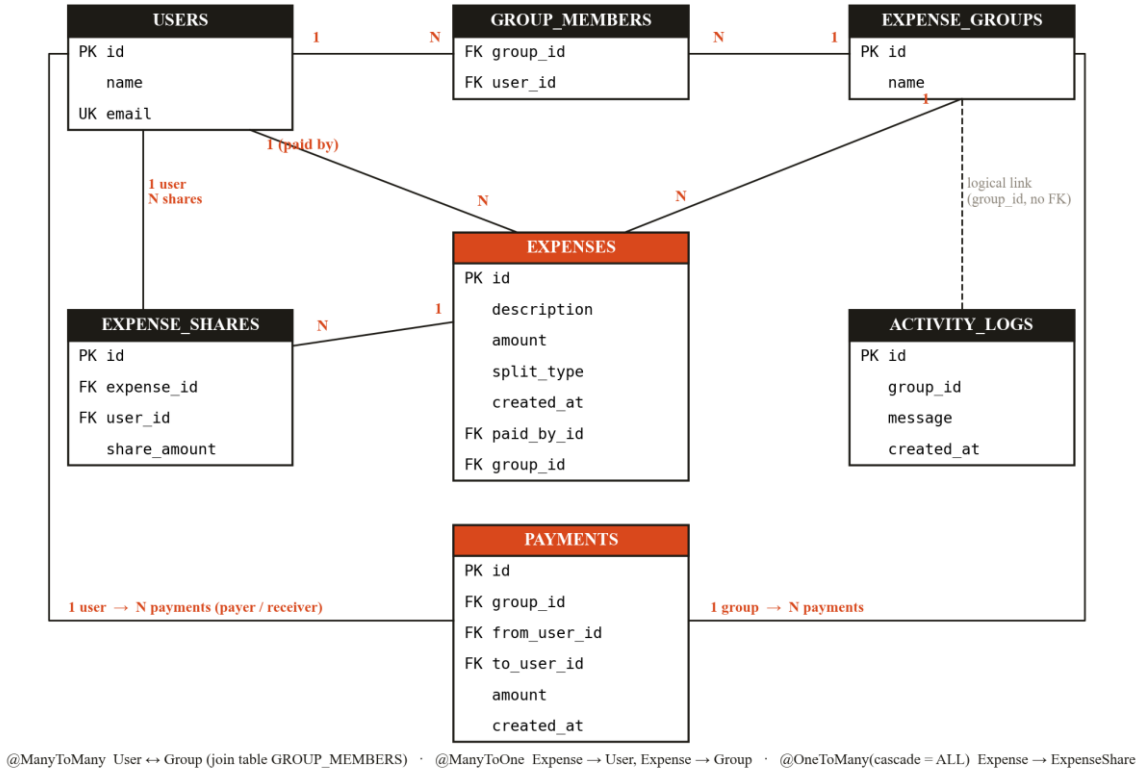


Figure 3.2 Entity-relationship diagram of the database

Figure 3.3 shows how the three design patterns are realised as classes, and Figure 3.4 the complete flow of adding an expense, including the points where the request is rejected and where the transaction is rolled back.

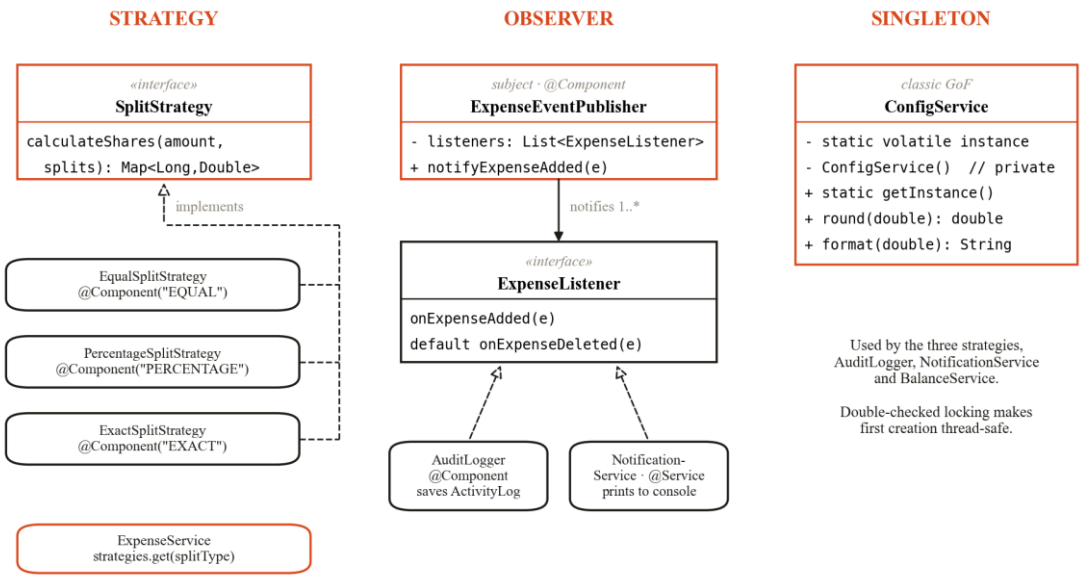


Figure 3.3 Class diagram of the Strategy, Observer and Singleton patterns

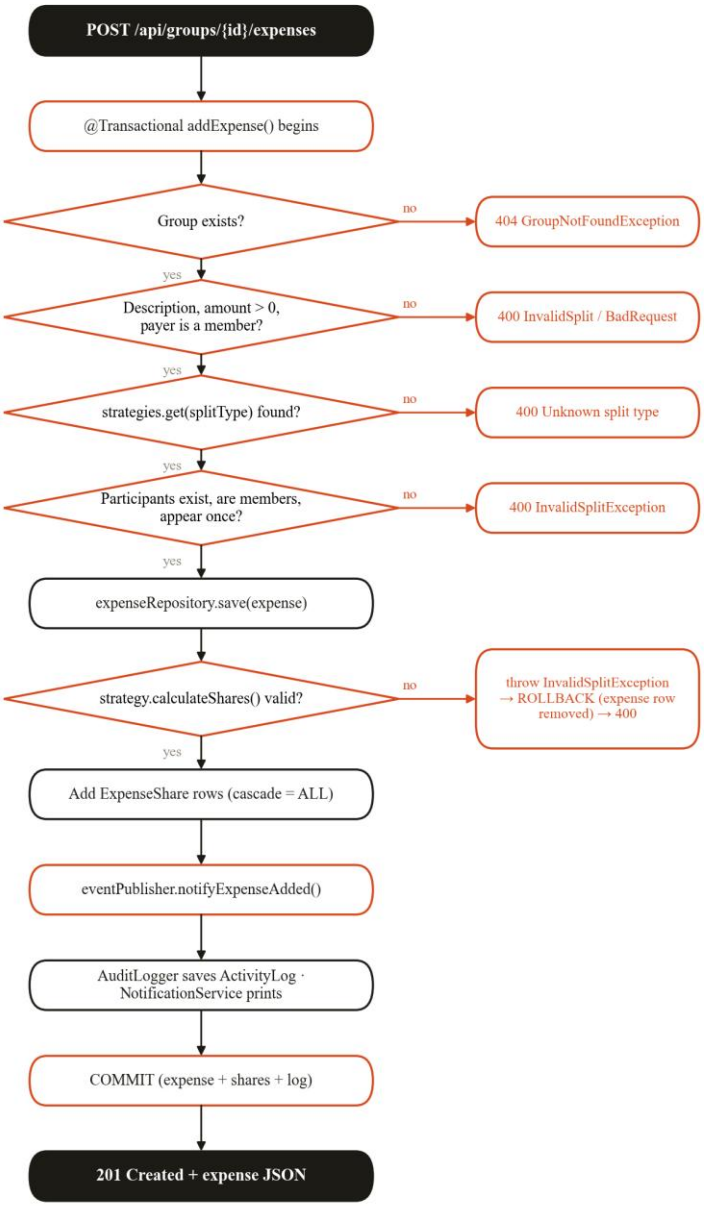


Figure 3.4 Flowchart of the transactional add-expense operation

Algorithm: computing balances and settlements

1. Load all expenses of the group with their shares, and all recorded payments.
2. paid[u] = total amount of expenses that member u paid; owed[u] = total of u's shares (Collectors.groupingBy with summingDouble, using flatMap over the shares).
3. sent[u] and received[u] = totals of settle-up payments made and received by u.
4. For every member, net = round(paid − owed + sent − received). Positive means the member gets money back, negative means the member owes, and $|net| \leq 0.01$ means settled. The nets always sum to zero.

5. Put every member with net > 0 into a max-heap of creditors and every member with net < 0 into a max-heap of debtors keyed by |net| (PriorityQueue with a lambda comparator [17]).
6. While both heaps are non-empty, remove the largest creditor C and the largest debtor D; D pays C the amount x = min(C, D), recorded as “D pays C Rs x”.
7. Subtract x from both; whoever still has a non-zero amount is pushed back into its heap.
8. Each iteration settles at least one member completely, so the loop ends after at most n − 1 payments, with a cost of O(n log n).

CHAPTER 4.
RESULTS ANALYSIS AND VALIDATION

4.1. Implementation of Solution

4.1.1. Tools and technologies

Table 4.1 Tools used in the project

Tool	Use
Java 17 (Temurin)	Implementation language: OOP, interfaces, generics, lambdas, Streams, collections
Spring Boot 3.3.5	Auto-configuration, embedded Tomcat, dependency injection
Spring Web	@RestController, request mapping, JSON via Jackson
Spring Data JPA, Hibernate 6	Object-relational mapping, JpaRepository, derived queries, transactions
H2 Database 2.2	In-memory database and web console (MySQL 8 optional)
Apache Maven 3.9.9 + Maven Wrapper	Build and dependency management; mvnw spring-boot:run
JUnit 5, Spring Boot Test	Integration test of the full scenario
Postman, Newman 6	API testing with automated assertions; command-line runs
HTML, CSS, JavaScript	Single-page web interface served by Spring Boot (no framework)
VS Code, Claude Code	Development environment and AI-assisted development
Matplotlib, Microsoft Word	Diagrams and report preparation

4.1.2. Code organisation

The backend consists of 44 Java source files (about 2,160 lines including comments) in nine packages, one JUnit test class, the application.properties file and an 890-line single-page web interface. **entity:** User, Group, Expense, ExpenseShare, Payment, ActivityLog. **repository:** six JpaRepository interfaces. **dto:** ExpenseRequest, SplitRequest, PaymentRequest, BalanceResponse, SettlementResponse, ErrorResponse. **strategy:** SplitStrategy and three implementations. **observer:** ExpenseListener, ExpenseEventPublisher, AuditLogger, NotificationService. **config:** ConfigService. **service:** UserService, GroupService, ExpenseService, BalanceService, PaymentService. **controller:** User, Group, Expense and Payment controllers. **exception:** six custom exceptions and the GlobalExceptionHandler.

4.1.3. Split strategies

Each split type is a Spring bean named after the type, so ExpenseService can look it up in its injected Map<String, SplitStrategy> with strategies.get(splitType). Table 4.2 summarises the rules.

Table 4.2 Split strategies

Split type	Value sent per person	Rule	Validation	Rounding
EQUAL	None	amount ÷ participants	At least one participant	Leftover paisa to the first person
PERCENTAGE	Percent	amount × percent ÷ 100	Values ≥ 0; total = 100 (±0.01)	Leftover to the last person
EXACT	Rupees	Value as given	Values ≥ 0; total = amount (±0.01)	Each value to 2 decimals

For example, Rs 1000 split equally among three people gives 333.34, 333.33 and 333.33, so the shares always add up to exactly the bill.

4.1.4. Transaction management and rollback

ExpenseService.addExpense() is annotated with @Transactional. It first saves the expense and only then asks the strategy to calculate the shares. If the split is invalid, the strategy throws InvalidSplitException, a RuntimeException, and Spring rolls back the whole transaction, so the expense row disappears again. The AuditLogger runs inside the same transaction, so the log entry is saved or discarded together with the expense. This was observed directly during testing: an expense rejected for percentages totalling 90 consumed ID 4 from the identity sequence, the next successful expense received ID 5, and the database (Figure 4.9) contains no expense 4.

4.1.5. Balance and settlement engine

Table 4.3 shows the balances for the demonstration scenario: a Rs 1200 dinner paid by Aman split equally, a Rs 1000 cab paid by Bhavya split 25/25/30/20 percent, and Rs 1000 of movie tickets paid by Chirag split exactly as 250/300/200/250.

Table 4.3 Balances for the demonstration scenario

Member	Paid (Rs)	Share owed (Rs)	Net balance (Rs)
Aman	1200	300 + 250 + 250 = 800	+400
Bhavya	1000	300 + 250 + 300 = 850	+150
Chirag	1000	300 + 300 + 200 = 800	+200
Diya	0	300 + 200 + 250 = 750	−750

The settlement algorithm returns three payments — “Diya pays Aman Rs 400.00”, “Diya pays Chirag Rs 200.00” and “Diya pays Bhavya Rs 150.00” — instead of the up to six pairwise payments a four-person group could need. When a member records a payment, for example Diya paying Aman Rs 250, the payment enters the balance formula, Diya’s

debt falls to Rs 500, and the next settlement suggests only the remaining Rs 150 to Aman (Figure 4.5).

4.1.6. REST API

Table 4.4 REST endpoints

Method	Endpoint	Purpose	Success	Errors
POST	/api/users	Create a user	201	400
GET	/api/users, /api/users/{id}	List users / get one	200	404
DELETE	/api/users/{id}	Delete a user	204	404, 409
POST	/api/groups	Create a group	201	400
GET	/api/groups, /api/groups/{id}	List groups / get one	200	404
DELETE	/api/groups/{id}	Delete a group and its data	204	404
POST	/api/groups/{g}/members/{u}	Add a member	200	400, 404
POST	/api/groups/{g}/expenses	Add an expense	201	400, 404
GET	/api/groups/{g}/expenses?userId=	List / filter expenses	200	404
DELETE	/api/expenses/{id}	Delete an expense	204	404
GET	/api/groups/{g}/balances	Net balance per member	200	404
GET	/api/groups/{g}/settlements	Suggested payments	200	404
POST	/api/groups/{g}/payments	Record a settle-up payment	201	400, 404
GET	/api/groups/{g}/payments	List payments	200	404
DELETE	/api/payments/{id}	Undo a payment	204	404
GET	/api/groups/{g}/activity	Activity log (observer)	200	404

All errors share one JSON format produced by GlobalExceptionHandler, for example {"status": 400, "error": "Bad Request", "message": "Percentages must add up to 100, but got 90.0"} together with a timestamp. Wrong HTTP methods return 405 and malformed IDs return 400 in the same format.

4.1.7. User interface

The single-page interface is served by the same Spring Boot application at http://localhost:8080 and calls the REST API with fetch(). It uses a ledger-style design with light and dark themes, and a console strip at the bottom shows every request with its method, URL, status code and JSON body. Figures 4.1 to 4.10 show the main screens and results.

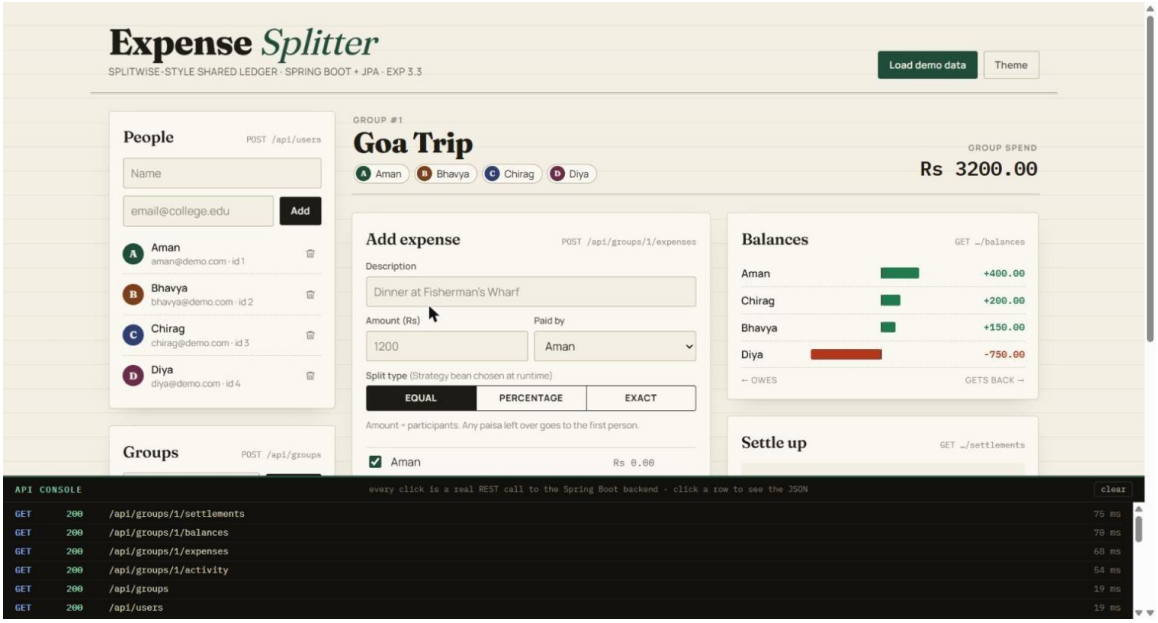


Figure 4.1 Dashboard after loading the demonstration data (light theme)

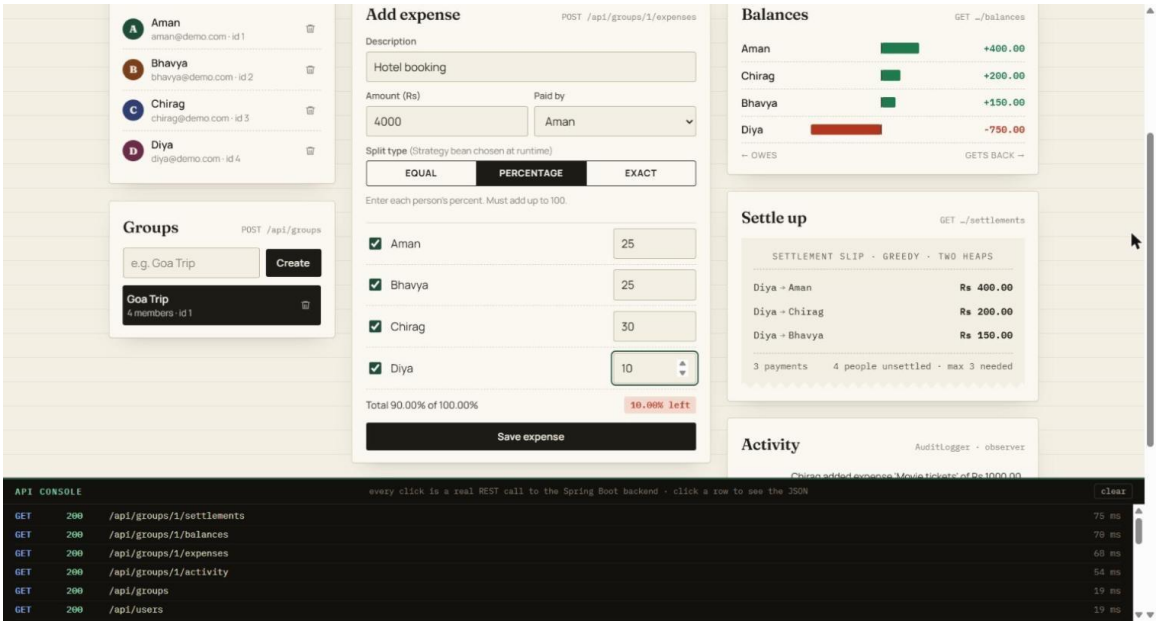


Figure 4.2 PERCENTAGE split with the live split meter showing 10% unallocated

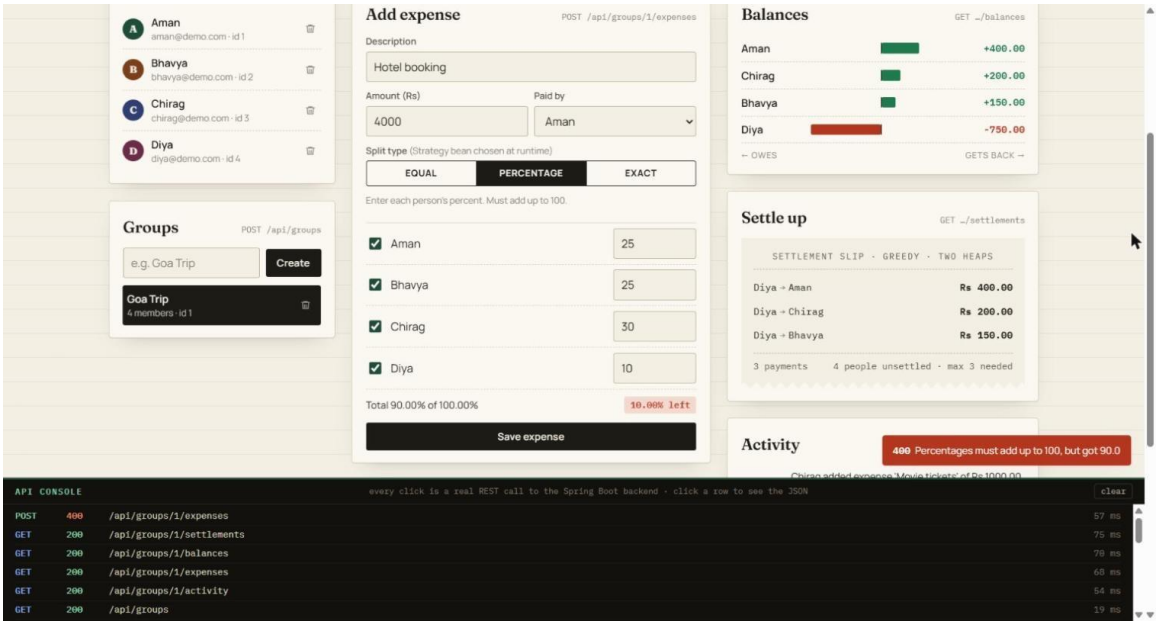


Figure 4.3 The server rejects the invalid split with 400 Bad Request

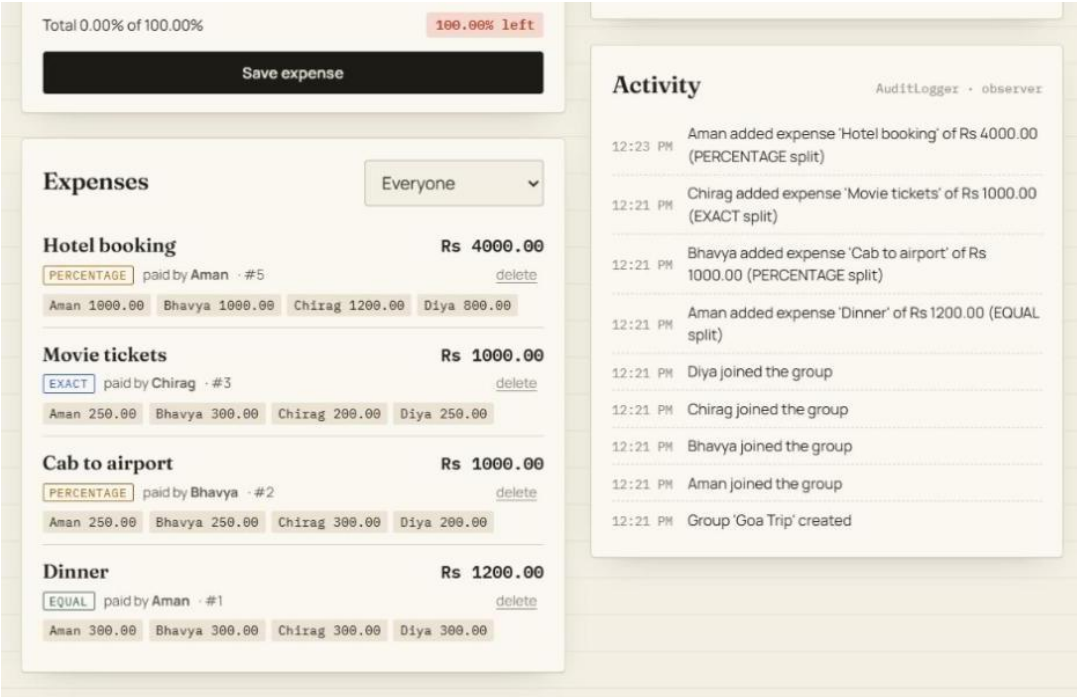


Figure 4.4 Expense list with per-member shares and the AuditLogger activity feed

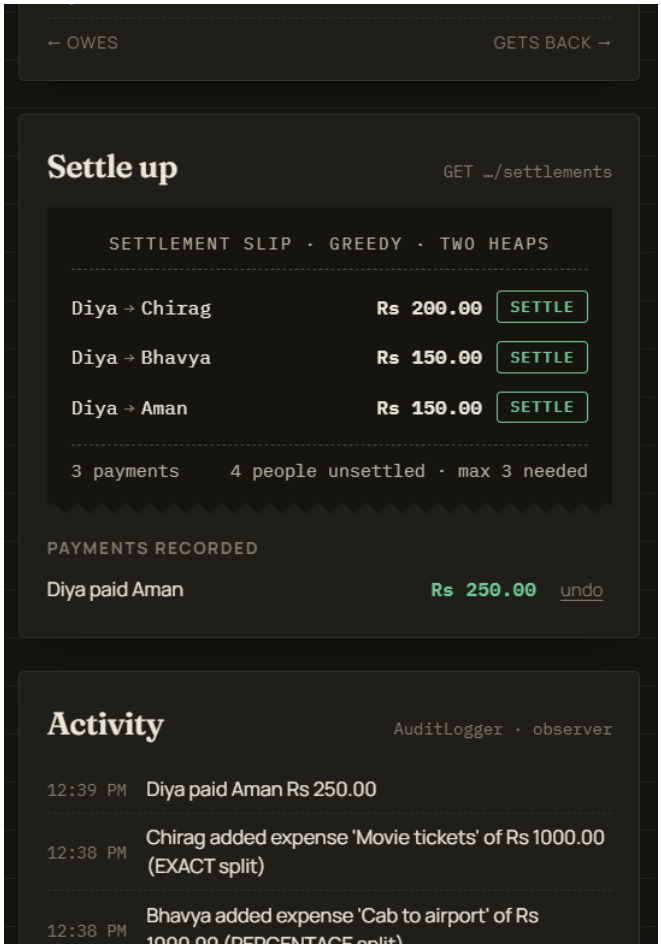


Figure 4.5 Settle-up slip after a partial payment, with Settle buttons and payment history

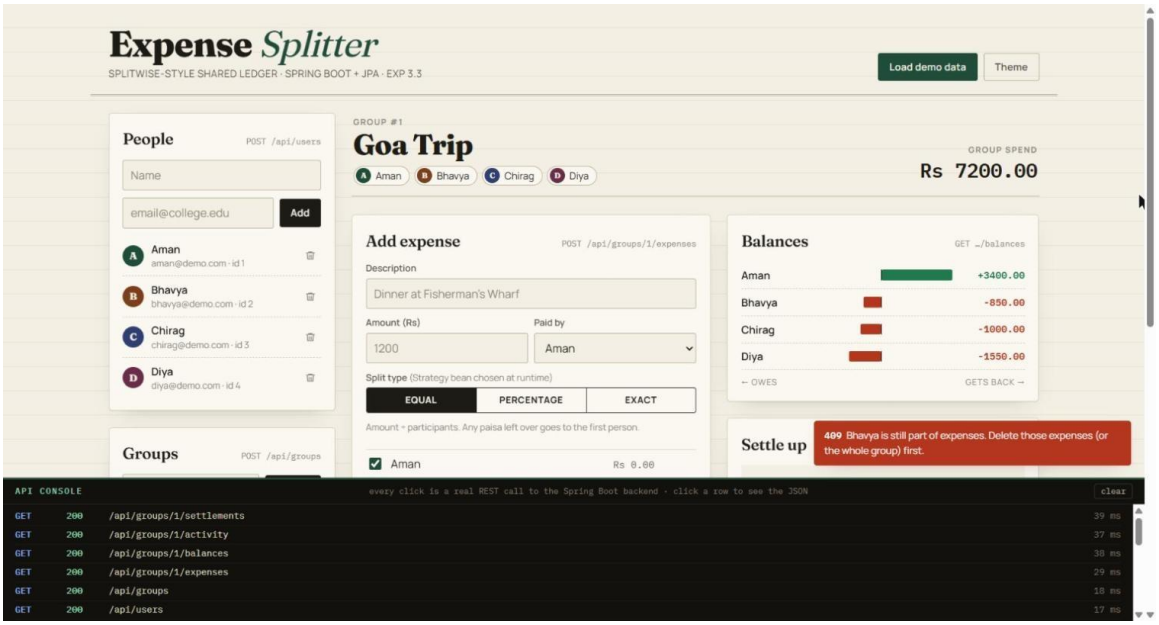


Figure 4.6 Deleting a member who still has expenses returns 409 Conflict

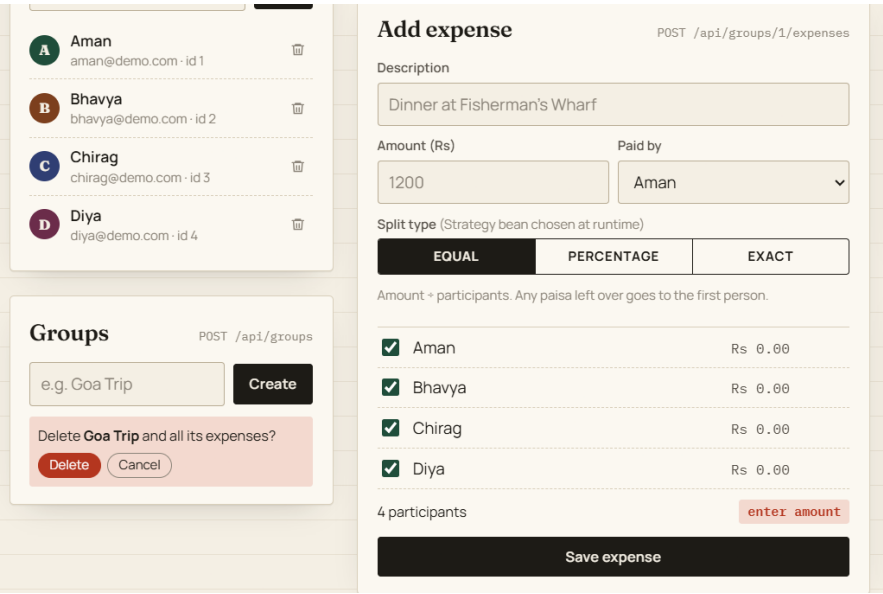


Figure 4.7 Inline confirmation before deleting a group

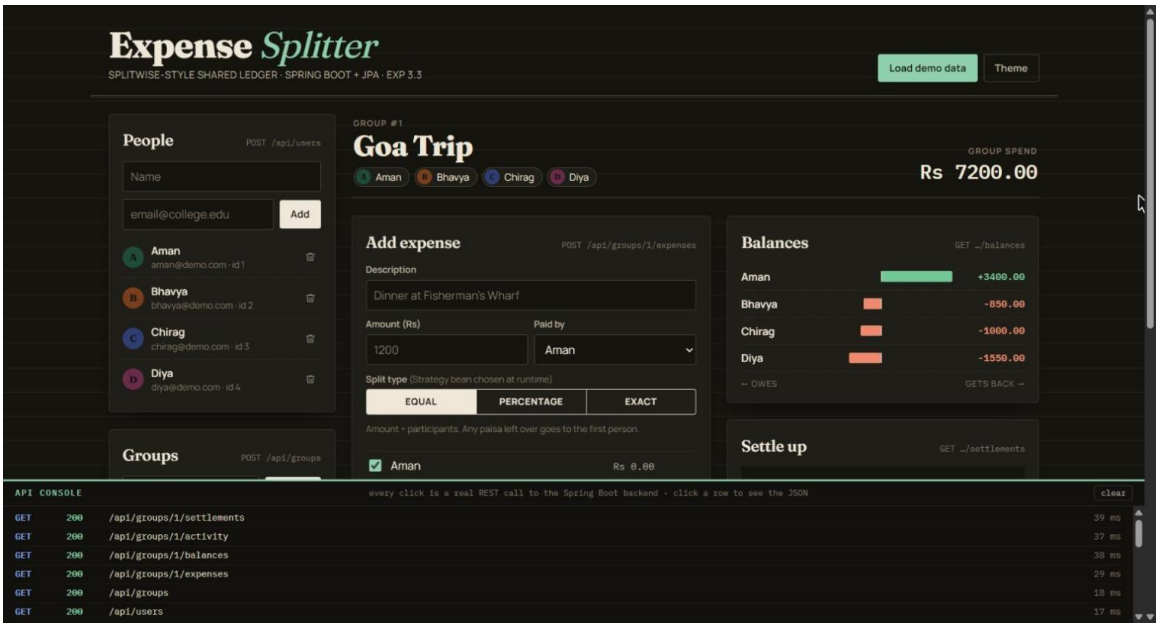


Figure 4.8 Dashboard in the dark theme

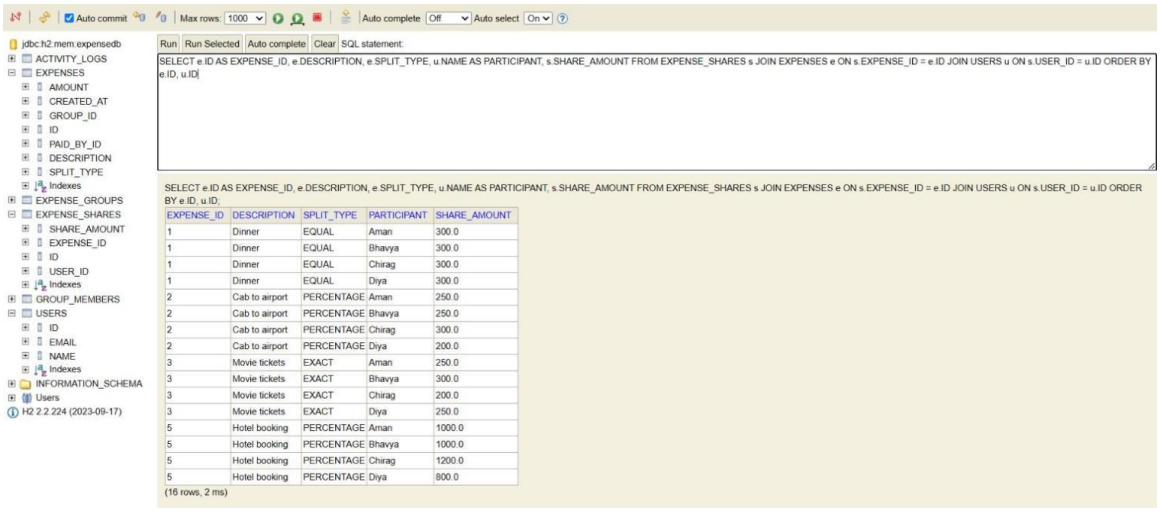


Figure 4.9 H2 console showing the expense shares stored by Hibernate

4.1.8. Testing and validation

The application was tested at three levels. A JUnit integration test (@SpringBootTest) runs the complete scenario in code and checks balances, settlements, the rollback of an invalid expense and settle-up payments. A Postman collection of 41 requests with 59 assertions, organised in eight folders, exercises every endpoint, including all error paths; it saves IDs from each response into collection variables and deletes everything it created, so it can be re-run at any time. The collection was executed with Newman, Postman's command-line runner (Figure 4.10). Finally, every screen of the web interface was used manually in both themes. Table 4.5 summarises the functional test cases.

Table 4.5 Functional test cases and results

#	Test case	Expected result	Result
1	Build and run JUnit test (mvnw test)	Compiles; full scenario passes	Pass
2	Start the application (mvnw spring-boot:run)	Tomcat on port 8080; tables created	Pass (7.8 s)
3	Create 4 users, a group, add members	201 / 201 / 200; group has 4 members	Pass
4	EQUAL Rs 1200 among 4	Four shares of Rs 300.00	Pass
5	PERCENTAGE Rs 1000 at 25/25/30/20	Shares 250 / 250 / 300 / 200	Pass
6	EXACT Rs 1000 at 250/300/200/250	Shares as given	Pass
7	Balances	+400, +200, +150, −750; sum is zero	Pass
8	Settlements	Three payments, all from Diya	Pass
9	Percentages totalling 90	400; no expense saved (rollback)	Pass
10	EXACT total ≠ amount; unknown split type	400 with explanatory message	Pass
11	Duplicate e-mail	400 "Email already registered"	Pass
12	Unknown group or user ID	404 with JSON error body	Pass
13	Delete an expense twice	204, then 404; settlements recalculated	Pass

#	Test case	Expected result	Result
14	Record a payment, then pay more than owed	201, then 400 “only owes …”	Pass
15	Undo a payment twice	204, then 404	Pass
16	Delete a user who still has expenses	409 Conflict	Pass
17	Delete a group, then its users	204 for all; group then 404	Pass
18	Wrong HTTP method / malformed ID	405 / 400 with JSON body	Pass
19	Full Postman collection with Newman	41 requests, 59 assertions, 0 failures	Pass (5 s run)

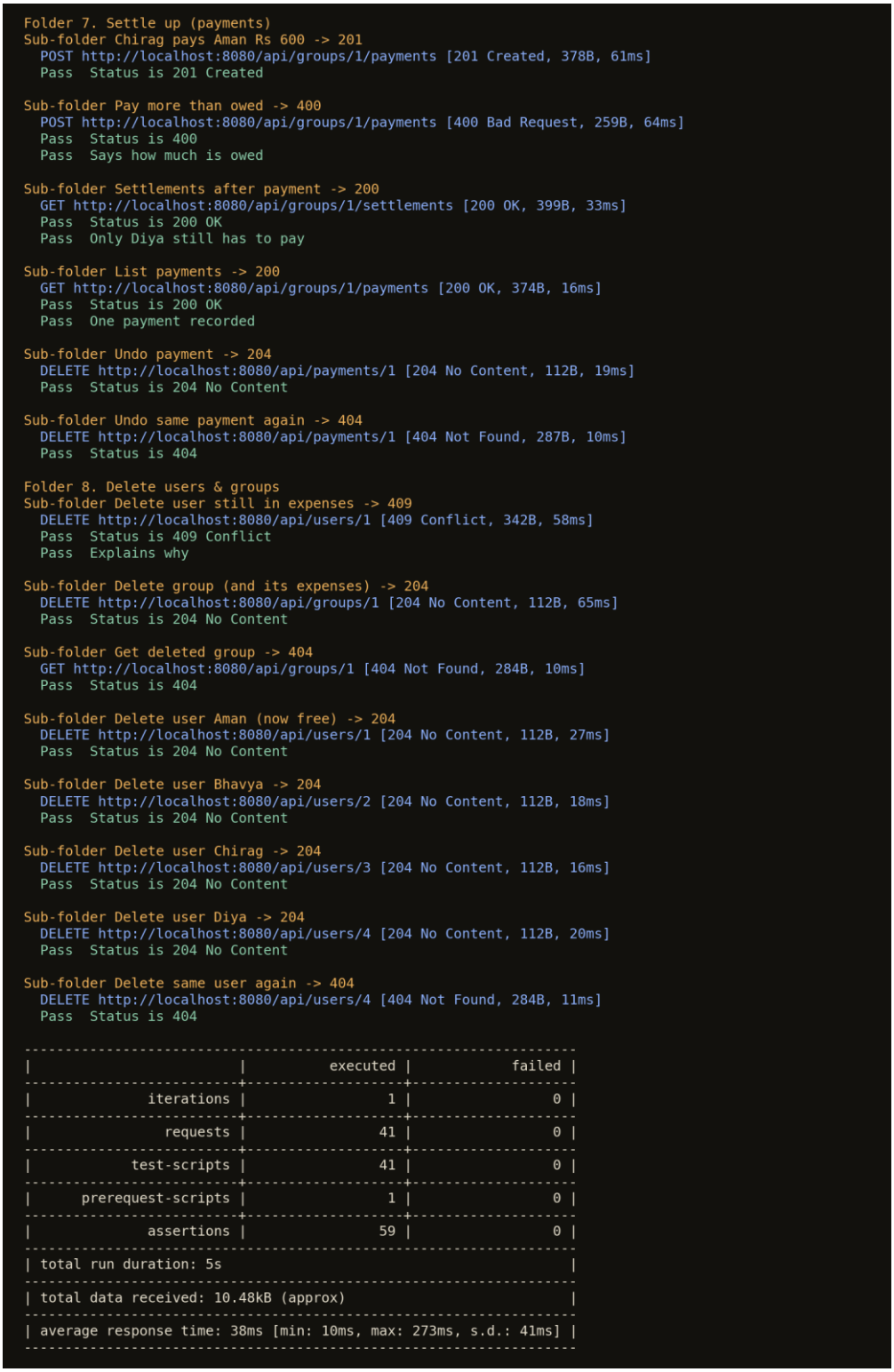


Figure 4.10 Newman run of the Postman collection (last two folders and summary)

Quality of the greedy settlement. Because the specification asked for the minimum number of transactions, the greedy algorithm was compared with an exact solver. The exact minimum equals n − k, where k is the largest number of disjoint groups of members whose balances sum to zero; this was computed by exhaustive search over subsets. Table 4.6

summarises the results for the project scenarios and for random groups with balances in multiples of Rs 50.

Table 4.6 Greedy settlement compared with the exact minimum

Case	Greedy payments	Minimum payments	Observation
Demonstration scenario	3	3	Optimal
After deleting the EXACT expense	3	3	Optimal
Balances +600, +400, +300, −800, −300, −200	5	4	Pairing +300/−300 first saves one payment
3,000 random groups of 4 members	100% optimal	—	Never worse
3,000 random groups of 5 members	97.8% optimal	—	At most 1 extra payment
3,000 random groups of 6 members	91.3% optimal	—	At most 1 extra payment
3,000 random groups of 7 members	81.3% optimal	—	At most 2 extra payments
1,500 random groups of 8 members	75.7% optimal	—	At most 2 extra payments

All functional test cases passed. The comparison confirms the literature [11]: the greedy method always stays within n − 1 payments and is optimal for typical small groups, but it is a heuristic, not a guarantee of the minimum. This is stated in the interface and documentation and motivates the exact-settlement item in future work.

CHAPTER 5.
CONCLUSION AND FUTURE WORK

5.1. Conclusion

Expense Splitter meets its objectives. It records group expenses with three interchangeable split strategies, saves each expense atomically, computes balances that always sum to zero, proposes a settlement of at most n − 1 payments, records partial settle-up payments with undo, and reports every error as clean JSON with the correct status code. The project visibly applies every concept required by the experiment — Spring Boot, REST controllers, JPA relationships, repositories, @Transactional, @ControllerAdvice, IoC/DI, Streams, lambdas, collections and the Strategy, Observer and Singleton patterns — and all 59 automated API assertions and the JUnit test pass.

The complete source code, build files, Postman collection, README, Postman test guide and viva cheat-sheet are in the project folder expense-splitter and run with a single command, mvnw spring-boot:run.

Deviations from the expected results and their reasons:

- The specification asked for the *minimum* number of transactions. Testing showed that the greedy algorithm guarantees at most n − 1 payments but not always the minimum (Table 4.6), so the claim was corrected in the code comments and documentation.
- The derived query findByGroupIdOrderByCreatedAtAsc failed at start-up because the helper getter getGroupId() made Spring Data look for a groupId attribute; it was renamed findByGroup_IdOrderByCreatedAtAsc to follow the relation explicitly.
- A two-click confirmation on the expense delete button made users believe deleting did not work, and a stale list could show an expense already deleted elsewhere (404). Delete became one click with an automatic refresh; only the destructive group delete keeps an explicit confirmation.
- Deleting users and groups and recording settle-up payments were added after the original specification, which required new integrity rules (409 for users still in expenses or payments, cascade for groups, and payments limited to the amount owed).

- The reserved SQL words user and group forced different table names (users, expense_groups).

5.2. Future Work

- **Exact settlement for small groups:** compute the true minimum by searching for the largest set of disjoint zero-sum subsets (dynamic programming over subsets) when $n \leq 15$, and keep the greedy method for larger groups.
- **Authentication and authorisation:** add Spring Security with JWT so that each user sees only their own groups.
- **BigDecimal end to end:** store amounts as DECIMAL(12,2) columns and use BigDecimal in the entities.
- **UPI deep links:** generate upi://pay links or QR codes for each suggested payment and mark the payment as recorded once it is confirmed.
- **More expense features:** multi-currency trips with exchange rates, recurring expenses and receipt photos.
- **Real notifications:** e-mail or push notifications as additional Observer listeners.
- **Deployment:** MySQL in Docker, and continuous integration that runs JUnit and Newman on every commit.
- **Mobile client:** an Android app that uses the same REST API.

REFERENCES

[1] National Payments Corporation of India, “Unified Payments Interface (UPI) – Product overview.” [Online]. Available: https://www.npci.org.in/what-we-do/upi/product-overview

[2] Splitwise, Inc., “Splitwise.” [Online]. Available: https://www.splitwise.com

[3] D. Goldberg, “What every computer scientist should know about floating-point arithmetic,” *ACM Computing Surveys*, vol. 23, no. 1, pp. 5–48, 1991.

[4] IEEE, *IEEE Standard for Binary Floating-Point Arithmetic*, ANSI/IEEE Std 754-1985, 1985.

[5] E. F. Codd, “A relational model of data for large shared data banks,” *Communications of the ACM*, vol. 13, no. 6, pp. 377–387, 1970.

[6] E. Gamma, R. Helm, R. Johnson, and J. Vlissides, *Design Patterns: Elements of Reusable Object-Oriented Software*. Reading, MA: Addison-Wesley, 1994.

[7] R. T. Fielding, “Architectural styles and the design of network-based software architectures,” Ph.D. dissertation, University of California, Irvine, 2000.

[8] M. Fowler, *Patterns of Enterprise Application Architecture*. Boston, MA: Addison-Wesley, 2002.

[9] R. Johnson, *Expert One-on-One J2EE Design and Development*. Indianapolis, IN: Wrox Press, 2002.

[10] M. Fowler, “Inversion of control containers and the dependency injection pattern,” Jan. 2004. [Online]. Available: https://martinfowler.com/articles/injection.html

[11] T. Verhoeff, “Settling multiple debts efficiently: An invitation to computing science,” *Informatics in Education*, vol. 3, no. 1, pp. 105–126, 2004.

[12] Java Community Process, “JSR 220: Enterprise JavaBeans 3.0 (including the Java Persistence API),” 2006.

[13] Spring, “Spring Boot reference documentation.” [Online]. Available: https://docs.spring.io/spring-boot/

[14] Spring, “Spring Framework reference documentation: Dependency injection.” [Online]. Available: https://docs.spring.io/spring-framework/reference/

[15] Spring, “Spring Data JPA reference documentation.” [Online]. Available: https://docs.spring.io/spring-data/jpa/reference/

[16] Hibernate, “Hibernate ORM documentation.” [Online]. Available: https://hibernate.org/orm/documentation/

[17] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to Algorithms*, 3rd ed. Cambridge, MA: MIT Press, 2009.

[18] Postman, Inc., “Newman: the command-line collection runner for Postman.” [Online]. Available: https://github.com/postmanlabs/newman

APPENDIX

A. Design Checklist

Table A.1 Design checklist

Item	Status	Where
Need justified with documented evidence	Done	Section 1.1
Problem stated without hinting at the solution	Done	Section 1.2
Literature reviewed with timeline and analysis	Done	Chapter 2
Measurable objectives defined	Done	Section 2.6
Constraints (cost, privacy, ethics, safety, standards) considered	Done	Section 3.2
At least two alternative designs compared	Done	Sections 3.4–3.5
Block diagram, ER diagram, flowchart and algorithm provided	Done	Section 3.6
Implementation shown with screenshots	Done	Section 4.1
Test cases executed and results recorded	Done	Tables 4.5–4.6
Source code, build files and test collection delivered	Done	Section 5.1

B. Project at a Glance

Table A.2 Technical summary of Expense Splitter

Item	Detail
Platform	REST backend with a browser interface (Windows, macOS, Linux)
Language / framework	Java 17, Spring Boot 3.3.5
Persistence	Spring Data JPA, Hibernate, H2 2.2 (MySQL optional)
Build / run	Maven Wrapper: mvnw spring-boot:run; interface at http://localhost:8080
Architecture	Controller–service–repository; 44 Java files, about 2,160 lines
Design patterns	Strategy (split types), Observer (expense events), Singleton (ConfigService)
API	18 REST endpoints; status codes 200, 201, 204, 400, 404, 405, 409
Money handling	BigDecimal HALF_UP rounding to 2 decimals; remainder assigned to one participant
Settlement	Greedy over two PriorityQueue max-heaps; at most n − 1 payments; O(n log n)
Testing	JUnit integration test; Postman collection of 41 requests and 59 assertions (Newman)

C. Key Source Code Excerpts

(a) Strategy beans injected as a Map and chosen at run time — ExpenseService.java

```
public ExpenseService(ExpenseRepository expenseRepository, GroupService
groupService,
                      UserService userService, ExpenseEventPublisher
eventPublisher,
                      Map<String, SplitStrategy> strategies) {   // constructor
injection
    ...
}

SplitStrategy strategy = strategies.get(splitType);   // "EQUAL", "PERCENTAGE" or
"EXACT"
if (strategy == null) {
    throw new InvalidSplitException("Unknown split type '" + request.getSplitType()
+ "'");
}
```

(b) All-or-nothing expense creation — ExpenseService.java

```
@Transactional
public Expense addExpense(Long groupId, ExpenseRequest request) {
    ...
    expense = expenseRepository.save(expense);                        // 1. save the
expense
    Map<Long, Double> shares = strategy.calculateShares(amount, splits); // 2. may
throw -> ROLLBACK
    shares.forEach((userId, share) -> expense.addShare(new
ExpenseShare(userService.getUser(userId), share)));
    eventPublisher.notifyExpenseAdded(expense);                      // 3. observers
(audit log)
    return expense;
}
```

(c) Net balances with Streams — BalanceService.java

```
Map<Long, Double> paid = expenses.stream()
        .collect(Collectors.groupingBy(e -> e.getPaidBy().getId(), HashMap::new,
                Collectors.summingDouble(Expense::getAmount)));
Map<Long, Double> owed = expenses.stream()
        .flatMap(e -> e.getShares().stream())
        .collect(Collectors.groupingBy(s -> s.getUser().getId(), HashMap::new,
                Collectors.summingDouble(ExpenseShare::getShareAmount)));
```

(d) Greedy settlement with two max-heaps — BalanceService.java

```
PriorityQueue<Party> creditors = new PriorityQueue<>((a, b) ->
Double.compare(b.amount, a.amount));
PriorityQueue<Party> debtors   = new PriorityQueue<>((a, b) ->
Double.compare(b.amount, a.amount));
while (!creditors.isEmpty() && !debtors.isEmpty()) {
    Party creditor = creditors.poll();
    Party debtor = debtors.poll();
    double pay = config.round(Math.min(creditor.amount, debtor.amount));
    result.add(new SettlementResponse(..., debtor.name + " pays " + creditor.name +
" " + config.format(pay)));
    creditor.amount = config.round(creditor.amount - pay);
    debtor.amount = config.round(debtor.amount - pay);
    if (creditor.amount > config.getTolerance()) creditors.add(creditor);
    if (debtor.amount > config.getTolerance()) debtors.add(debtor);
}
```

(e) Thread-safe Singleton — ConfigService.java

```
private static volatile ConfigService instance;
private ConfigService() { ... }                 // nobody can call new
ConfigService()

public static ConfigService getInstance() {
    if (instance == null) {
        synchronized (ConfigService.class) {
            if (instance == null) instance = new ConfigService();
        }
    }
    return instance;
}
```

D. Sample API Requests and Responses

(a) Add a PERCENTAGE expense — POST /api/groups/1/expenses

```
{
  "description": "Cab to airport", "amount": 1000, "paidBy": 2, "splitType":
"PERCENTAGE",
  "splits": [ {"userId": 1, "value": 25}, {"userId": 2, "value": 25},
              {"userId": 3, "value": 30}, {"userId": 4, "value": 20} ]
}
```

(b) Error response when the percentages do not total 100 — 400 Bad Request

```
{ "timestamp": "2026-10-22T12:00:27.92", "status": 400, "error": "Bad Request",
  "message": "Percentages must add up to 100, but got 90.0" }
```

(c) Settlement for the demonstration scenario — GET /api/groups/1/settlements → 200 OK

```
[ { "fromUser": "Diya", "toUser": "Aman",   "amount": 400.0, "message": "Diya pays
Aman Rs 400.00" },
  { "fromUser": "Diya", "toUser": "Chirag", "amount": 200.0, "message": "Diya pays
Chirag Rs 200.00" },
  { "fromUser": "Diya", "toUser": "Bhavya", "amount": 150.0, "message": "Diya pays
Bhavya Rs 150.00" } ]
```

(d) Record a partial settle-up payment — POST /api/groups/1/payments → 201 Created

```
{ "fromUserId": 4, "toUserId": 1, "amount": 250 }
```

EXPENSE SPLITTER: A SPLITWISE-STYLE GROUP EXPENSE SETTLEMENT SYSTEM USING SPRING BOOT AND DESIGN PATTERNS

Submitted by

Parikshit (24BET10298)
Dhruv Gouniyal (24BET10295)
Sagar Makkar (24BET10296)
Manan (24BET10199)

Under the supervision of

Harsh Gupta, Assistant Professor
Department of Information Technology, Chandigarh University

Project folder: expense-splitter · run with mvnw spring-boot:run

— End of Report —